



OPEN Automatically optimizing heuristics for robust scale-free network design via large language models

He Yu^{1,2}✉ & Jing Liu^{1,2}

Achieving scale-free robust networks is a challenging problem due to its NP-hard nature and the vast, complex, high-dimensional solution space. Current methods, from handcrafted feature extraction to deep learning approaches, have made certain advancements but remain rigid and complex, often requiring manual design, trial and error, and large amounts of labeled data. To address these challenges, we propose AutoRNet, a novel framework that integrates large language models (LLMs) with evolutionary algorithms to generate complete heuristics for robust network design, supported by expert-crafted Network Optimization Strategies (NOSs). With the intrinsic properties of robust scale-free network structures in mind, NOS-based variation operations provide domain-specific prompts for LLMs, leveraging expert-defined domain knowledge to guide the creation of advanced heuristics. Moreover, to address the difficulty posed by the hard constraint of maintaining degree distributions, an adaptive fitness function is designed, progressively strengthening constraints to balance convergence and diversity. We evaluate the robustness of networks generated by AutoRNet's heuristics on both sparse and dense initial scale-free networks. These solutions outperform those from current methods. AutoRNet reduces the extent of manual heuristic design by utilizing structured domain knowledge through expert-crafted NOSs, offering a flexible and adaptive approach for generating robust scale-free network structures. The source code and dataset for this work have been made publicly available at <https://github.com/leonyuhe/AutoRNet>.

Keywords Evolutionary algorithms, Large language models, Complex networks, Network robustness, Prompt engineering, Deep learning

Modern networked systems, particularly scale-free networks characterized by power-law degree distributions, form the backbone of contemporary society^{1–3}. These networks, prevalent in real-world systems such as social, biological, and internet infrastructures, exhibit unique robustness properties, including resilience to random failures but vulnerability to targeted attacks⁴. Improving the robustness of scale-free networks to prevent catastrophic disruptions is inherently challenging due to the NP-hard nature of the problem and the complex, high-dimensional solution space. Although research in this area has made significant advancements, key contributions include approximate theoretical models⁵, which simplify the complex interactions within scale-free networks to provide insights for enhancing resilience. Optimization algorithms, employing approaches such as simulated annealing (SA)⁶, genetic algorithms (GAs)⁷, and greedy approaches⁸, offer near-optimal solutions in a reasonable time frame. In addition, machine learning techniques, particularly deep reinforcement learning⁹, have been used to adapt dynamically and improve network configurations. However, these methods depend greatly on manual work, expert knowledge, training data, and trial-and-error processes.

The emergence of coding-oriented large language models (LLMs)¹⁰ has garnered significant attention because of their potential in addressing combinatorial optimization problems^{11,12}. FunSearch¹³ combines a pretrained LLM with evolutionary algorithms (EAs) to evolve initial low-scoring programs into high-scoring ones. Evolution of heuristics (EoH)¹⁴ is applied to evolve heuristic descriptions and their corresponding code implementations together. However, both FunSearch and EoH depend primarily on existing optimization algorithms, and only scoring or weighting mechanisms that influence the search algorithm behavior are designed by LLMs to guide the related operations without creating a new algorithm. This limits the applicability of these methods to complex domain-specific problems, such as ensuring network robustness. *Therefore, there is a need to generate whole algorithms directly rather than merely modifying data through weighting and scoring.*

¹School of Artificial Intelligence, Xidian University, 2 South Taibai Road, Xi'an 710071, Shaanxi, China. ²Guangzhou Institute of Technology, Xidian University, Knowledge City, Guangzhou 510555, Guangdong, China. ✉email: yuhe001@stu.xidian.edu.cn

In this paper, we address these issues and make a significant contribution by proposing AutoRNet, an integrated framework that combines the contextual intelligence of LLMs with the adaptive optimization capabilities of EAs. Specifically, we design network optimization strategy (NOS)-based variation operations tailored for complex network problems, which can create domain-specific prompts for LLMs, generating a variety of heuristics that are suitable for different network-related challenges. Moreover, we design an adaptive fitness function (AFF) to evaluate these heuristics, progressively tightening the constraints to balance convergence and diversity, thereby obtaining superior heuristics. AutoRNet leverages special network characteristics, through expert-crafted NOSs, to guide the design of effective operations. The major contributions of this paper are summarized as follows:

- A hybrid framework is developed, in which EAs and LLMs iteratively collaborate to generate and refine heuristics for optimizing network robustness.
- NOS-based variation operations are designed, utilizing expert-defined prompts to guide LLMs in divergent thinking. These NOSs are equally applicable to other network-related challenges.
- An AFF tailored to network issues is designed to transform hard constraints into soft constraints, increasing the diversity and resilience of heuristics.
- The solutions generated by AutoRNet's heuristics are evaluated across eight scale-free networks with varying sizes and densities and a real-world network, and it is shown that they outperform current methods.

The remainder of this paper is organized as follows: Sections “[Related work](#)” and “[Network robustness measures](#)” introduce related work and network robustness measures, respectively. Section “[AutoRNet](#)” provides a detailed overview of AutoRNet. Section “[Experiments](#)” presents the experiments, followed by “[Limitations and future research directions](#)”, which explores the limitations and future research directions. Finally, “[Conclusion](#)” concludes the paper with a summary of our findings.

Related work

Related work on improving network robustness

Traditional methods for improving network robustness focus on improving local redundancy and connectivity, such as by maximizing the *clustering coefficient* to form tightly knit groups of nodes. These methods also leverage high-order network structures such as triangles and quadrilaterals to identify and protect *critical nodes* and *edges* to withstand targeted attacks. Techniques such as *greedy algorithms* and *local search algorithms* systematically increase network robustness. Chan et al.¹⁵ introduced a local search algorithm by iteratively improving the subgraph structure to improve the robustness of the network.

Metaheuristic algorithms, including *genetic algorithms* (GAs)¹⁶ and *simulated annealing* (SA)¹⁷, have also been extensively employed to solve network robustness problems. Pizzuti et al.¹⁸ proposed using GAs to enhance the robustness of complex networks by simulating the process of natural selection to evolve network configurations over generations, improving resilience against failures and attacks. Zhou et al.⁷ proposed the use of memetic algorithms, which combine global and local search strategies, to increase the robustness of scale-free networks by integrating global and local search operators. Buesser et al.⁶ proposed the use of SA to optimize the robustness of scale-free networks by rewiring the network edges, thus improving the resilience of the network to fragmentation and intentional damage.

However, these methods often require manual design and multiple trial-and-error processes, making them inefficient and time-consuming. The need for extensive experimentation and tuning reduces their practicality in large-scale and dynamic network environments, highlighting the need for more automated and adaptive approaches to network robustness optimization.

Deep learning, particularly *graph neural networks* (GNNs), has emerged as a powerful approach for addressing network robustness problems. Tang et al.¹⁹ explored methods of increasing the robustness of GNNs against poisoning attacks by leveraging clean graphs from similar domains. This approach helps the GNNs detect adversarial edges more effectively, enhancing their resistance to attacks. Wang et al.²⁰ developed certifiably robust GNNs to defend against attacks that perturb the graph structure by adding or deleting edges. Their approach ensures that the performance of the GNNs remains stable even under adversarial conditions.

Despite their strengths, the above approaches face significant challenges. Traditional and metaheuristic methods often require manual design and extensive trial-and-error, making them inefficient for large-scale or dynamic networks. Although GNN-based methods of increasing network robustness are powerful, they require large amounts of labeled data for training, which can be difficult to obtain in practice. The high computational demand of training and inference is also a limitation, especially in real-time applications. Moreover, GNNs can still be vulnerable to sophisticated adversarial attacks, where small, carefully crafted perturbations in the input data can lead to incorrect classifications. These limitations underscore the need for automated, adaptive methods like AutoRNet to optimize network robustness efficiently.

Application of LLMs in combinatorial optimization

LLMs have had a significant impact on addressing various problems. One prevalent approach involves *engineering in-context learning prompts* (EILP)²¹ via techniques such as *zero-shot*, *few-shot*, and *chain-of-thought* (CoT) *prompting*^{22–24}. Enhancing EILP with *fine-tuning* further increases the response accuracy by adapting LLMs to specific datasets or tasks. Furthermore, combining EILP with *ensemble learning* techniques increases robustness and consistency by aggregating multiple model predictions. Integration with *retrieval-augmented generation* (RAG)²⁵ enables the use of external knowledge sources, allowing LLMs to generate more accurate and informed contextual responses.

Many researchers have also focused on solving combinatorial optimization problems via LLMs. Liu et al.²⁶ presented the first investigation into LLMs as evolutionary combination optimizers for solving the travelling

salesman problem (TSP). FunSearch¹³ leverages LLMs to generate and optimize mathematical functions, discover new constructions and improve existing solutions by iteratively refining the function constructions. EoH¹⁴ facilitates the simultaneous evolution of concept descriptions and code implementations, emulating the process by which humans develop heuristics to achieve efficient automatic heuristic design. These approaches have demonstrated competitive performance compared with that of traditional heuristics in finding high-quality solutions.

All of these methods are based on existing algorithms; scoring or weighting mechanisms are used to modify the original data (e.g., bin capacities, city distances), influencing the search algorithm behaviour without creating a new algorithm. *There is a need to generate whole algorithms directly rather than merely modifying data through weighting and scoring.* The optimization improvements of these approaches are restricted to relatively simple combinatorial problems, and *they cannot be extended to more complex or domain-specific optimization scenarios.*

Unlike FunSearch, which evolves scoring functions within predefined algorithmic skeletons, and EoH, which co-evolves heuristic descriptions and code for simpler combinatorial problems, AutoRNet generates complete heuristics tailored to the domain-specific challenge of network robustness. By integrating NOSs into LLM prompts, AutoRNet leverages intrinsic network properties like degree distributions and centrality measures to guide heuristic design. Furthermore, AutoRNet's AFF dynamically balances hard constraints, overcoming the limitations of rigid constraint enforcement and enhancing heuristic diversity and effectiveness. Benefiting from the NOS and AFF, AutoRNet can design sophisticated algorithms that generalize across diverse network topologies, outperforming traditional and prior LLM-based methods in synthetic and real-world scenarios.

Network robustness measures

A network is often represented as a graph $G = (V, E)$, where V denotes the set of nodes and E represents the set of edges. To measure network robustness, several methodologies have been developed. Literature³ introduced a measure R to assess network robustness by evaluating the size of the largest connected component during sequential node attacks:

$$R = \frac{1}{N} \sum_{q=1}^N s(q) \quad (1)$$

where N is the number of nodes in the network and $s(q)$ is the fraction of nodes in the largest connected cluster after removing q nodes. The normalization factor $1/N$ ensures that the robustness of networks of different sizes can be compared. The range of R values is between $1/N$ and 0.5.

Complex networks can exhibit different topological structures, with random and scale-free networks being two of the most studied types. Random networks, characterized by a homogeneous degree distribution, are robust to targeted attacks on high-degree nodes. In contrast, scale-free networks, with a power-law degree distribution, exhibit exceptional robustness against random failures due to the low probability of removing a hub but are extremely susceptible to targeted attacks that focus on their few high-degree hubs. These networks have distinct characteristics that significantly impact their robustness and vulnerability to failures or attacks.

In this paper, we use R to evaluate network robustness, specifically focusing on targeted attacks through the sequential removal of the highest-degree nodes. We employ scale-free networks for both training and testing graph sets to accurately reflect the structures of many real-world networks.

AutoRNet Framework of AutoRNet

AutoRNet uses EAs to search heuristics and maintains a population of *popsize* individuals, denoted as $P = \{h_1, h_2, \dots, h_{popsize}\}$. Each individual h_i , $i = 1, 2, \dots, popsize$, includes a heuristic. There are T generations in total, where $P_t = \{h_{t,1}, h_{t,2}, \dots, h_{t,popsize}\}$ represents the population at generation t . AutoRNet evolves the population generation by generation, and the whole framework is summarized in Algorithm 1.

Input: Training graph set: $\mathcal{G}$
Parameter: Population size: *popsize*, Number of generations: T , Variation probabilities: p_{e1} , p_{m1} , and p_{m2}
Output: *bestIndividual*

```

1:  $P_1 \leftarrow \text{InitializePopulation}(\text{popsize})$ 
2:  $\text{AFF}(P_1, \mathcal{G}, 1)$ 
3: for  $t = 1$  to  $T$  do
4:    $P_{\text{offspring}} \leftarrow \text{NOS\_Variation}(P_t, p_{e1}, p_{m1}, p_{m2})$ 
5:    $\text{AFF}(P_{\text{offspring}}, \mathcal{G}, t)$ 
6:    $P_{t+1} \leftarrow \text{SelectNextPopulation}(P_t, P_{\text{offspring}})$ 
7: end for
8:  $\text{bestIndividual} \leftarrow \text{FindBestIndividual}(P_T)$ 
9: return bestIndividual

```

Algorithm 1. AutoRNet

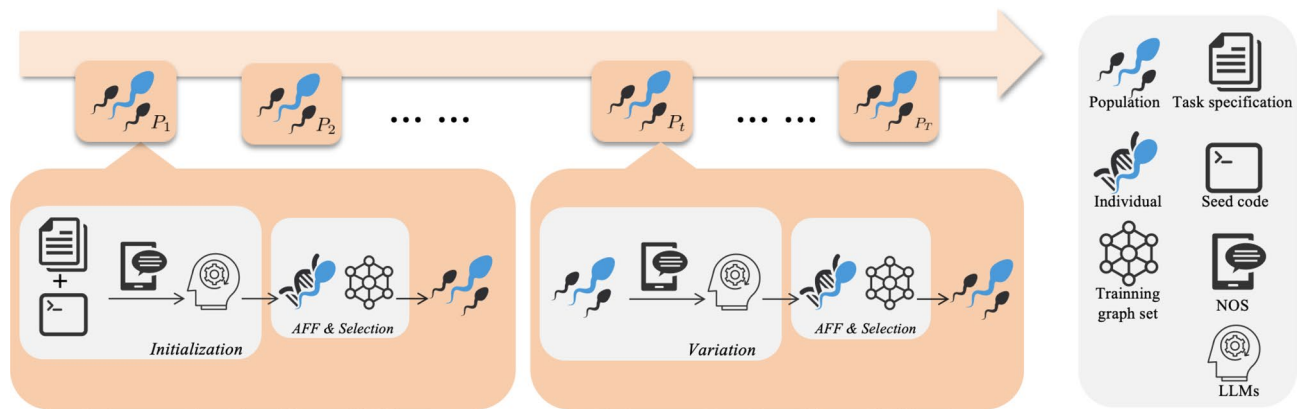


Figure 1. A schematic illustration of AutoRNet.

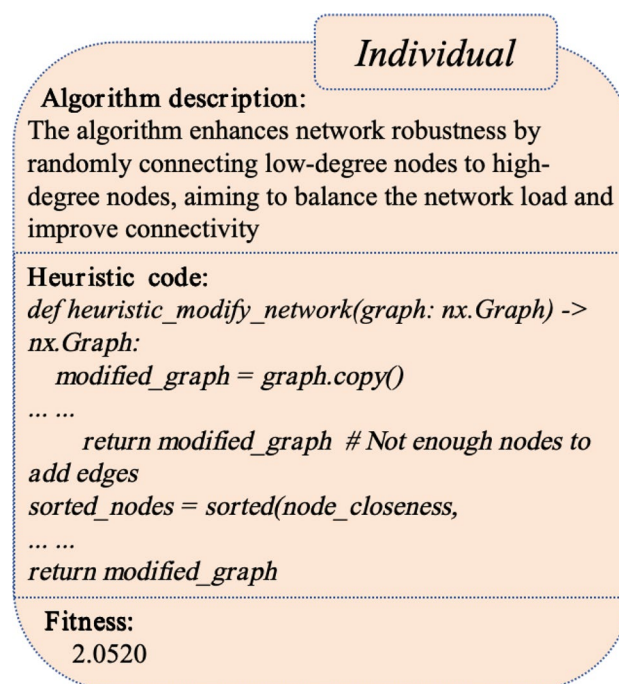


Figure 2. Individual structure schematic.

First, *InitializePopulation()* sets the initial population P_1 of *popsiz*e individuals using a *task specification* and prompt interaction with the LLM. Then, on the basis of the training graph set $\mathcal{G}$, *AFF()* uses the adaptive fitness function introduced below to calculate the fitness of each individual in P_1 . Next, the population is evolved for T generations, and in each generation, *NOS_Variation()* first applies the NOS-based variation operations described below to the current population, obtaining the offspring population. Then, *SelectNextPopulation()* uses roulette wheel selection according to fitness to select *popsiz*e individuals from P_t and $P_{offspring}$ together to form the population for the next generation. Figure 1 schematically illustrates the framework of AutoRNet.

Individual encoding and population initialization

The individual's heuristic codes are generated by LLMs. Each individual is structured as described below and schematically illustrated in Fig. 2:

- **Algorithm Description:** A natural-language explanation of the heuristic's objective, providing context and clarification regarding what the heuristic aims to achieve.
- **Heuristic Code:** The actual implementation of the heuristic, detailing the logic and operations needed to improve network robustness.
- **Fitness:** The fitness of the heuristic as evaluated by *AFF*.

AutoRNet initializes its heuristic population by providing LLMs with a detailed *task specification* that includes *a description of the problem, function signature, and functionality of the heuristic*. The LLM generates the initial heuristics by creating and implementing them as Python code, with the process repeated multiple times to ensure diversity. Optionally, *heuristic seed codes*, either simple or expert-designed, are included to guide the LLM towards effective methods. Existing hand-crafted heuristics can also be incorporated to enhance the initial population with additional domain knowledge. The detailed population initialization prompts are provided in Appendix A.

Adaptive fitness function

When using an EA to assist LLMs in heuristic code generation, individuals are encoded as heuristics—methods that can generate solutions rather than problem solutions—operating in a distinct search space with a mapping to solution fitness, and the search is based on LLMs that generate *new or similar* methods. *This does not inherently provide a mechanism for measuring the similarity between methods, making it difficult to navigate the method space*. Maintaining a consistent degree distribution is a constraint in optimizing network robustness. Strictly enforcing this constraint from the outset can be overly restrictive, leading to many invalid individuals and thereby impeding the evolutionary process.

To address these issues, we design an *adaptive fitness function (AFF)*, which can dynamically adjust the degree distribution constraint throughout the evolutionary process. The AFF initially relaxes this constraint, facilitating broader exploration of the method space, and then progressively tightens this constraint as the evolution continues. In traditional EAs, individuals represent problem solutions, and selection methods like roulette wheel or tournament selection preserve diversity by probabilistically including suboptimal solutions based on their fitness. AFF relaxes constraints directly on heuristics during selection, enabling a diverse heuristic population to evolve. Over generations, these constraints tighten to ensure valid, robust solutions, offering a valuable approach to designing effective heuristics for complex network optimization.

Fitness computation

The AFF evaluates heuristics on a training graph set $\mathcal{G} = \{G_1, \dots, G_M\}$ of Barabási-Albert scale-free graphs. By applying low penalties for structural deviations (e.g., degree distribution) early and increasing them later, AFF fosters diverse heuristic exploration before converging to constraint-compliant solutions.

For heuristic $h_{t,i}$ at generation t , its function $H_{t,i}$ optimizes $\mathcal{G}$ to $\tilde{\mathcal{G}}_{t,i} = \{\tilde{G}_{t,i,1}, \dots, \tilde{G}_{t,i,M}\}$:

$$H_{t,i}(\mathcal{G}) = \tilde{\mathcal{G}}_{t,i} \quad (2)$$

The fitness of $h_{t,i}$ is calculated as:

$$f(\tilde{\mathcal{G}}_{t,i}, t) = \sum_{j=1}^M \left(R(\tilde{G}_{t,i,j}) \cdot (2 - w(t) \cdot Y(\tilde{G}_{t,i,j})) \right) \quad (3)$$

where $R(\tilde{G}_{t,i,j})$ measures robustness, and $Y(\tilde{G}_{t,i,j})$ quantifies deviation:

$$Y(\tilde{G}_{t,i,j}) = \frac{D_{\text{diff}}(\tilde{G}_{t,i,j})}{D_{\text{max}}(\tilde{G}_{t,i,j})} + \frac{E_{\text{diff}}(\tilde{G}_{t,i,j})}{E_{\text{max}}(\tilde{G}_{t,i,j})} \quad (4)$$

where the degree deviation is:

$$D_{\text{diff}}(\tilde{G}_{t,i,j}) = \frac{1}{N} \sum_{k=1}^N |d_k(G_j) - d_k(\tilde{G}_{t,i,j})| \quad (5)$$

with maximum:

$$D_{\text{max}}(\tilde{G}_{t,i,j}) = \max_{k \in \{1, \dots, N\}} |d_k(G_j) - d_k(\tilde{G}_{t,i,j})| \quad (6)$$

The edge deviation is:

$$E_{\text{diff}}(\tilde{G}_{t,i,j}) = |E(G_j) - E(\tilde{G}_{t,i,j})| \quad (7)$$

with maximum:

$$E_{\text{max}}(\tilde{G}_{t,i,j}) = \max(E(G_j), E(\tilde{G}_{t,i,j})) \quad (8)$$

The penalty weight $w(t)$ grows as:

$$w(t) = \left(\frac{t}{T} \right)^p \quad (9)$$

where T is total generations, and $p \in [0.5, 2]$.

Evolutionary impact

The AFF’s adaptive process, starting with relaxed constraints and progressively increasing penalties via $w(t)$, fundamentally enhances AutoRNet’s ability to generate a diverse set of heuristics. Early in the evolution, minimal penalties allow heuristics with varying structural deviations to thrive, fostering exploration across the heuristic space and preventing premature convergence to local optima. As $w(t)$ grows, the AFF guides selection toward heuristics that meet constraints, achieving maximum fitness ($2 \times R(\tilde{G}_{t,i,j})$) when deviations ($D_{\text{diff}} = E_{\text{diff}} = 0$) are eliminated. This dynamic adjustment ensures a rich diversity of heuristics initially, which is critical for discovering innovative strategies, while later refining them into robust, valid solutions for diverse network topologies.

Network optimization strategy-based variation operations

The heuristic-method search space for EAs assisting LLMs in heuristic code generation is vast, complex, and high-dimensional. In simpler problem domains, such as those addressed by EoH and FunSearch, this space is reduced by limiting function codes to straightforward tasks such as weighting or scoring data. Complex problems such as network robustness cannot be simplified because of the intricate and domain-specific nature of tasks. Through experiments, we find that in solving complex problems such as network robustness, current LLMs can only design simple operations on nodes or links because they are only provided with general prompts and not domain knowledge; they cannot make deep use of domain knowledge to design advanced operations. To overcome this, the *network optimization strategies* are designed with the intrinsic properties of networks in mind to guide LLMs in designing sophisticated heuristics.

Network optimization strategies

NOSs utilize network *features* such as degree distributions, clustering coefficients, centrality measures, and path characteristics to inform heuristic design. These features inspire *strategies* like prioritizing high-degree nodes, prioritizing betweenness centrality or optimizing shortest paths, which guide *actions* such as adding, rewiring, or swapping edges. *This combination of features, strategies, and actions forms the NOS.* The details are provided in Appendix B.

Variation operations

In AutoRNet, three variation operations—E1, M1, and M2—are designed to generate offspring heuristics by embedding NOSs in LLM prompts. E1 and M1 use 12 randomly selected NOSs with general prompts, while M2 relies on general prompts alone. Table 1 summarizes their roles.

- *E1 (Exploration with NOS Integration):* E1 prompts the LLM with two parent heuristics and 12 random NOSs to generate diverse offspring, fostering innovation and escaping local optima.
- *M1 (Guided Modification with NOSs):* M1 refines heuristics by prompting the LLM with NOSs, enabling targeted improvements through domain-guided local search.
- *M2 (Local Adjustment):* M2 prompts the LLM for minor heuristic tweaks without NOSs, optimizing performance via pure local search.

Each generation employs tournament selection to allocate individuals to E1, M1, and M2 based on probabilities p_{e1} , p_{m1} , and p_{m2} , selecting $p_{e1} \times \text{popsize}$, $p_{m1} \times \text{popsize}$, and $p_{m2} \times \text{popsize}$ individuals, respectively. Prompts are processed concurrently by the LLM server, enhancing efficiency. Figure 3 illustrates the M1 prompt schematic, with E1 and M2 details in Appendix C. These operations collectively balance exploration and refinement, optimizing network robustness.

Experiments

AutoRNet generates heuristic methods, and we analyse these methods to illustrate the design capabilities of AutoRNet. During the evolutionary phase, heuristics are evaluated on smaller-scale graphs to compute fitness scores efficiently, and in the testing phase, the finalized heuristics are applied to larger graphs, including real-world networks, to assess their generalizability and compare them against baseline algorithms. We select three existing algorithms as baselines: the hill climbing algorithm (HC)²⁷, the simulated annealing algorithm (SA)⁶, and the smart rewiring algorithm (SR)²⁸. R is used to evaluate the robustness of the networks in the test graph set after optimization by all the algorithms. By comparing their network robustness values, we assess the effectiveness of the methods generated by AutoRNet.

Experimental settings

The training graph set $\mathcal{G}$ consists of BA scale-free networks of two different sizes: 50 and 100 nodes. For each network size, the number of initial nodes M_0 varies from 2 to 5. For each combination of network size and M_0 ,

Operation	Purpose	Mechanism	Evolutionary Role
E1	Explore novel heuristics	Generates new heuristics with 12 random NOSs and 2 parents	Promotes diversity and escapes local optima
M1	Refine existing heuristics	Enhances heuristics via NOS-guided LLM prompts	Improves effectiveness with domain knowledge
M2	Optimize incrementally	Adjusts heuristics slightly without NOSs	Fine-tunes performance via local search

Table 1. Roles of variation operations in AutoRNet.

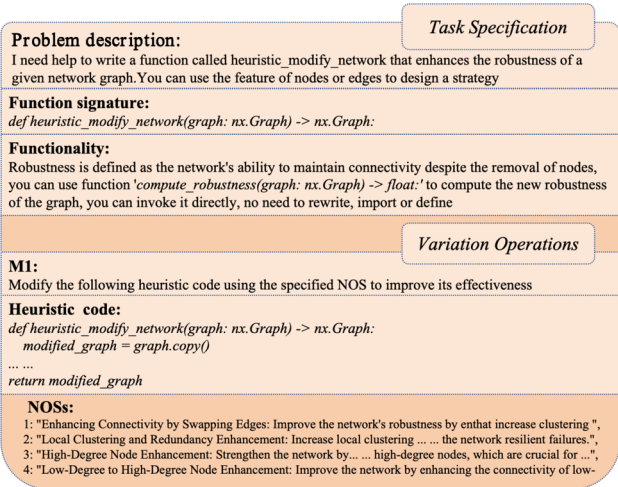


Figure 3. Variation operation prompt for M1.

Component	Runtime (seconds)	Cost (USD)
Heuristic Generation (500 LLM Calls)	250	\$3.50
Fitness Evaluations (50 Generations)	600	–
Total Training	850	\$3.50

Table 2. Computational costs for AutoRNet training (heuristic generation processes 200,000 tokens (125,000 input, 75,000 output) at \$0.01 per 1000 input tokens and \$0.03 per 1000 output tokens.).

we create three instances, resulting in a total of $M = 2 \times 4 \times 3 = 24$ training graphs. The test graph set consists of three types of networks: sparse BA networks with 100, 200, 300, and 500 nodes, $N_0 = 3$, and $M_0 = 2$; BA networks with 100 nodes, $N_0 = 6$, and M_0 ranging from 2 to 5; and a real-world EU power grid network²⁹ with 1,494 nodes and 2,066 edges.

Each heuristic method performs a series of network modifications, such as edge addition, relocation, and swapping. After each modification, R is used to evaluate the network's robustness. If R improves, the modification is accepted; otherwise, it is rolled back. Consequently, the R function is called multiple times to guide the optimization process. The total number of such evaluations for each heuristic is defined as $max_attempts$. For the training phase, $max_attempts$ is set to 100. For the testing phase, $max_attempts$ is set to 3×10^4 for BA networks and 5×10^4 for the EU power grid network, which is the same as those used in the three baseline algorithms. The experiments involving networks with 100 to 300 nodes are conducted over 100 independent runs; for networks with 500 nodes, over 30 independent runs are conducted; and for the EU power grid network, over 10 independent runs are conducted. The experiments are conducted on a dual-socket Intel Xeon Gold 6326 server with 32 cores, a base frequency of 2.90 GHz, and a maximum turbo frequency of 3.50 GHz, equipped with 256GB DDR4 REG ECC memory.

The parameters of AutoRNet are configured as follows: $popsiz$, T , p_{e1} , p_{m1} , and p_{m2} are set to 10, 50, 0.8, 0.1, and 0.1, respectively. The GPT-4 Turbo model is utilized with temperature settings of 1.0 and 0.5, and p in Equation 9 is set to 1.5. The seed heuristic employed in the experiment modifies the network graph through random edge swaps, as described in Appendix A.

Computational demands

AutoRNet's reliance on LLMs and repeated fitness evaluations results in notable computational demands during the training phase. Heuristic generation involves 10 LLM calls per generation for a population of 10 individuals across 50 generations, totaling 500 calls using GPT-4 Turbo. These calls are processed concurrently, with each batch of 10 calls generating Python heuristics (e.g., edge swap logic) from a prompt with NOSs, processing approximately 400 tokens (250 input, 150 output) and taking 5 seconds on average per batch. This results in a total heuristic generation time of 250 seconds (4.2 minutes) across 50 generations. As GPT-4 Turbo is accessed remotely via an API, local resource usage for heuristic generation is minimal, with computational costs driven by the remote service. The estimated token cost for 500 calls (200,000 tokens) is \$3.50, based on API pricing of \$0.01 per 1000 input tokens and \$0.03 per 1000 output tokens as detailed in Table 2.

Fitness evaluations based on R are performed locally. For the training phase, each of the 24 BA networks (50 and 100 nodes) undergoes 100 attempts per heuristic. Each individual's evaluations, processed concurrently across the 24 graphs, take approximately 0.5 seconds, and with 10 individuals per generation processed concurrently, each generation requires approximately 12 seconds, leveraging parallel processing across 32 cores. Across 50 generations, this results in 600 seconds (10 minutes) for fitness evaluations per training run. The

total training time, combining heuristic generation and fitness evaluations, is approximately 850 seconds (14.2 minutes) per run, as summarized in Table 2.

Evaluation results

Design capabilities of AutoRNet

After running AutoRNet for 50 generations, we select three highly valuable heuristics based on their fitness values, named *Heuristic-v1*, *Heuristic-v2* and *Heuristic-v3*, which significantly improve the network robustness of the test graph set. *Heuristic-v1* leverages the network features of critical nodes and similar nodes, *Heuristic-v2* utilizes the node connectivity and degree distribution, and *Heuristic-v3* optimizes the local topology by manipulating edges among neighbours. All three heuristics maintain the same number of edges, and *Heuristic-v1* also preserves the degree distribution. This demonstrates AutoRNet's ability to design complete algorithms that effectively utilize network features.

Heuristic-v1, shown in Algorithm 2, employs an advanced strategy that combines edge swapping with SA while preserving the degree distribution. This heuristic identifies critical nodes (those with the highest degrees) and pairs similar nodes on the basis of their degree deviation. It dynamically adjusts the *max_diff* parameter (Step 4), which controls the tolerance for degree deviation in pairing similar nodes (Steps 22–34). The algorithm iteratively swaps edges between the neighbours of these paired nodes (Steps 18–19), evaluating new configurations on the basis of robustness improvements or probabilistic acceptance criteria derived from simulated annealing (Step 22). This sophisticated approach ensures a balance between exploration and exploitation in the search space. Without NOSs, it would be impossible to evolve such “intelligent” heuristics to leverage network features for robust network design.

Input: Graph G , Integer $max_attempts$

<pre> 1: $modified_graph \leftarrow G.copy()$ 2: $initial_robustness \leftarrow compute_robustness(modified_graph)$ 3: $current_robustness \leftarrow initial_robustness$ 4: $initial_diff, max_diff, critical_percentage \leftarrow 2, 2, 0.1$ 5: $initial_temp, final_temp \leftarrow 1.0, 0.1$ 6: $critical_nodes \leftarrow identify_critical_nodes(modified_graph, critical_percentage)$ 7: for $attempt \in \{0, \dots, max_attempts - 1\}$ do 8: $temperature \leftarrow simulated_annealing_temperature(attempt, max_attempts, initial_temp, final_temp)$ 9: $similar_pairs \leftarrow find_similar_nodes(modified_graph, max_diff)$ 10: $node1, node2 \leftarrow random.choice(similar_pairs)$ 11: if $node1 \notin critical_nodes$ or $node2 \notin critical_nodes$ then 12: continue 13: end if 14: $neighbors1 \leftarrow list(modified_graph.neighbors(node1))$ 15: $neighbors2 \leftarrow list(modified_graph.neighbors(node2))$ 16: $neighbor1 \leftarrow random.choice(neighbors1)$ 17: $neighbor2 \leftarrow random.choice(neighbors2)$ 18: if $node1 \neq neighbor2$ and $node2 \neq neighbor1$ and $neighbor1 \neq neighbor2$ then </pre>	<pre> 19: $temp_graph \leftarrow swap_edges(modified_graph, node1, neighbor1, node2, neighbor2)$ 20: $new_robustness \leftarrow compute_robustness(temp_graph)$ 21: $delta_robustness \leftarrow new_robustness - current_robustness$ 22: if $delta_robustness > 0$ or $random.random() < exp(delta_robustness/temperature)$ then 23: $modified_graph \leftarrow temp_graph$ 24: if $delta_robustness > 0$ then 25: $current_robustness \leftarrow new_robustness$ 26: end if 27: $max_diff \leftarrow initial_diff$ 28: else 29: $max_diff \leftarrow max_diff + 1$ 30: end if 31: end if 32: if $attempt \bmod (max_attempts/10) == 0$ then 33: $max_diff \leftarrow initial_diff + random.randint(-2, 2)$ 34: end if 35: end for 36: return $modified_graph$ </pre>
--	---

Algorithm 2. Heuristic-v1: Enhancing Network Robustness via simulated Annealing and Edge Swaps

Heuristic-v2 and *Heuristic-v3* optimize the network through edge relocation, which, while slightly altering the degree distribution, significantly improves the robustness of the networks. Importantly, edge-relocation actions incur the same real-world costs as edge-swapping actions. *This shows that AutoRNet is not constrained by theoretical conditions but instead explores a variety of methods, making it more suitable for solving real-world problems.*

Heuristic-v2, shown in Algorithm 3, improves the network robustness by redistributing edges between high-degree and low-degree nodes. This heuristic involves adding an edge between a high-degree node and a low-degree node while removing an edge from the high-degree node.

Heuristic-v3, shown in Algorithm 4, increases robustness by redistributing edges among the neighbours of high-degree nodes. This method identifies high-degree nodes and strategically adds edges between their neighbours, maintaining the overall edge count but improving resilience to failure.

Input: Graph G , Integer $max_attempts$

```

1:  $modified\_graph \leftarrow G.copy()$ 
2:  $initial\_robustness \leftarrow compute\_robustness(modified\_graph)$ 
3:  $current\_robustness \leftarrow initial\_robustness$ 
4: for  $attempt \in \{0, \dots, max\_attempts - 1\}$  do
5:    $avg\_degree \leftarrow \sum(degrees.values())/len(degrees)$ 
6:    $high\_degree\_nodes \leftarrow$ 
     [for node in  $degrees.items()$  if  $degree > avg\_degree$ ]
7:    $low\_degree\_nodes \leftarrow$ 
     [for node in  $degrees.items()$  if  $degree < avg\_degree$ ]
8:   if not  $high\_degree\_nodes$  or not  $low\_degree\_nodes$  then
9:     continue
10:  end if
11:   $high\_node \leftarrow random.choice(high\_degree\_nodes)$ 
12:   $low\_node \leftarrow random.choice(low\_degree\_nodes)$ 
13:  if not  $modified\_graph.has\_edge(high\_node, low\_node)$  then
14:     $neighbors \leftarrow list(modified\_graph.neighbors(high\_node))$ 
15:    if not  $neighbors$  then
16:      continue
17:    end if
18:     $neighbor \leftarrow random.choice(neighbors)$ 
19:     $modified\_graph.remove\_edge(high\_node, neighbor)$ 
20:     $modified\_graph.add\_edge(high\_node, low\_node)$ 
21:     $new\_robustness \leftarrow compute\_robustness(modified\_graph)$ 
22:    if  $new\_robustness > current\_robustness$  then
23:       $current\_robustness \leftarrow new\_robustness$ 
24:    else
25:       $modified\_graph.remove\_edge(high\_node, low\_node)$ 
26:       $modified\_graph.add\_edge(high\_node, neighbor)$ 
27:    end if
28:  end if
29: end for
30: return  $modified\_graph$ 

```

Algorithm 3. Heuristic-v2: Redistributing Edges between High and Low-Degree Nodes

Input: Graph G , Integer $max_attempts$

```

1:  $modified\_graph \leftarrow G.copy()$ 
2:  $initial\_robustness \leftarrow compute\_robustness(modified\_graph)$ 
3:  $current\_robustness \leftarrow initial\_robustness$ 
4: for  $attempt \in \{0, \dots, max\_attempts - 1\}$  do
5:    $avg\_degree \leftarrow \sum(degrees.values())/len(degrees)$ 
6:    $high\_degree\_nodes \leftarrow$ 
     [for node in  $degrees.items()$  if  $degree > avg\_degree$ ]
7:    $node \leftarrow random.choice(high\_degree\_nodes)$ 
8:    $neighbors \leftarrow list(modified\_graph.neighbors(node))$ 
9:    $neighbor1, neighbor2 \leftarrow random.sample(neighbors, 2)$ 
10:  if not  $modified\_graph.has\_edge(neighbor1, neighbor2)$  then
11:     $modified\_graph.add\_edge(neighbor1, neighbor2)$ 
12:    if  $random() < 0.5$  then
13:       $edge\_to\_remove \leftarrow (node, neighbor1)$ 
14:    else
15:       $edge\_to\_remove \leftarrow (node, neighbor2)$ 
16:    end if
17:     $modified\_graph.remove\_edge(*edge\_to\_remove)$ 
18:     $new\_robustness \leftarrow compute\_robustness(modified\_graph)$ 
19:    if  $new\_robustness > current\_robustness$  then
20:       $current\_robustness \leftarrow new\_robustness$ 
21:    else
22:       $modified\_graph.remove\_edge(neighbor1, neighbor2)$ 
23:       $modified\_graph.add\_edge(*edge\_to\_remove)$ 
24:    end if
25:  end if
26: end for
27: return  $modified\_graph$ 

```

Algorithm 4. Heuristic-v3: Redistributing Edges among the Neighbors of High-Degree Nodes

N	Algs.	Best	Worst	Average ± Variance
100	HC	0.276153	0.213245	0.251327 ± 0.000211
	SA	0.307163	0.221321	0.274463 ± 0.000269
	SR	0.286163	0.211427	0.255401 ± 0.000229
	Heuristic-v1	0.274800	0.262700	0.270066 ± 0.000069
	Heuristic-v2	0.366399	0.355299	0.360699 ± 0.000020
	Heuristic-v3	0.324999	0.324599	0.324766 ± 0.000011
200	HC	0.279601	0.214745	0.247127 ± 0.000141
	SA	0.281831	0.227268	0.257501 ± 0.000089
	SR	0.277896	0.220101	0.253213 ± 0.000091
	Heuristic-v1	0.264925	0.254250	0.260791 ± 0.000058
	Heuristic-v2	0.354849	0.348724	0.351099 ± 0.000021
	Heuristic-v3	0.315075	0.303625	0.307550 ± 0.000028
300	HC	0.264859	0.223717	0.243451 ± 0.000064
	SA	0.265958	0.227663	0.249812 ± 0.000091
	SR	0.267519	0.230698	0.251128 ± 0.000063
	Heuristic-v1	0.249233	0.244899	0.247299 ± 0.000042
	Heuristic-v2	0.355855	0.348866	0.351699 ± 0.000027
	Heuristic-v3	0.309299	0.303022	0.307023 ± 0.000022
500	HC	0.249601	0.222911	0.236645 ± 0.000038
	SA	0.249788	0.221655	0.238325 ± 0.000028
	SR	0.249093	0.228155	0.238423 ± 0.000033
	Heuristic-v1	0.233875	0.227803	0.230369 ± 0.000033
	Heuristic-v2	0.348228	0.344426	0.346460 ± 0.000024
	Heuristic-v3	0.305000	0.301996	0.303977 ± 0.000023

Table 3. R values obtained for networks of different sizes.

M ₀	Algs.	Best	Worst	Average ± Variance
2	HC	0.267699	0.182465	0.231189 ± 0.000272
	SA	0.301401	0.195479	0.251081 ± 0.000367
	SR	0.270889	0.188913	0.234173± 0.000269
	Heuristic-v1	0.266000	0.260500	0.262601 ± 0.000132
	Heuristic-v2	0.346599	0.312231	0.333293 ± 0.000068
	Heuristic-v3	0.310200	0.302000	0.306233 ± 0.000088
3	HC	0.361012	0.284971	0.331501 ± 0.000214
	SA	0.377793	0.315786	0.357099 ± 0.000132
	SR	0.364866	0.291878	0.338351 ± 0.000182
	Heuristic-v1	0.359299	0.358399	0.358866 ± 0.000041
	Heuristic-v2	0.402699	0.385199	0.394033 ± 0.000051
	Heuristic-v3	0.387299	0.379999	0.382566 ± 0.000063
4	HC	0.400000	0.365644	0.386593 ± 0.000059
	SA	0.413366	0.385940	0.401635 ± 0.000033
	SR	0.402673	0.370594	0.388876 ± 0.000030
	Heuristic-v1	0.400299	0.395199	0.397299 ± 0.000025
	Heuristic-v2	0.308147	0.302105	0.305465 ± 0.000014
	Heuristic-v3	0.416999	0.410799	0.412866 ± 0.000021
5	HC	0.425451	0.399378	0.412409 ± 0.000024
	SA	0.436733	0.410297	0.423445 ± 0.000017
	SR	0.421089	0.400198	0.412844 ± 0.000022
	Heuristic-v1	0.420999	0.419599	0.420266 ± 0.000005
	Heuristic-v2	0.338199	0.331899	0.335066 ± 0.000004
	Heuristic-v3	0.429299	0.426099	0.427933 ± 0.000005

Table 4. R values obtained for networks with varying edge densities.

Algs.	Best	Worst	Average $\pm$ Variance
HC	0.213618	0.210541	0.212316 $\pm$ 0.000001
SA	0.215369	0.195734	0.206183 $\pm$ 0.000067
SR	0.214041	0.212116	0.212764 $\pm$ 0.000001
Heuristic-v1	0.211907	0.195907	0.205911 $\pm$ 0.000058
Heuristic-v2	0.272040	0.270839	0.271440 $\pm$ 0.000002
Heuristic-v3	0.220805	0.217966	0.219386 $\pm$ 0.000002

Table 5. *R* values obtained for the EU Power Grid Network.

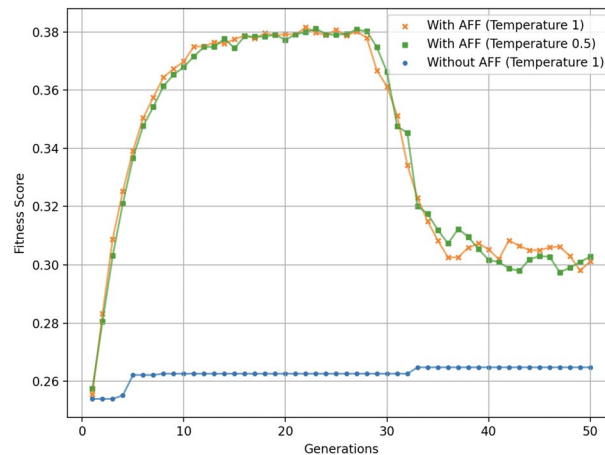


Figure 4. Fitness progression over 50 generations under three configurations: (1) without the AFF, (2) with the AFF at temperature = 1.0, and (3) with the AFF at temperature = 0.5.

The ability of AutoRNet to design heuristics that either strictly adhere to or flexibly navigate among *AFF* constraints indicates its versatility. AutoRNet not only matches the complexity of manually designed algorithms but also explores new strategies that yield better performance at the same practical cost. This leads to broader considerations regarding the potential of automated heuristic design to optimize complex networks in the real world.

Performance comparison of algorithms

The results over all test graphs, summarized in Tables 3, 4 and 5, demonstrate the effectiveness of the heuristics designed by AutoRNet.

Heuristic-v1's performance is not as strong as those of *Heuristic-v2* and *Heuristic-v3*, but it is comparable to those of the three manually designed algorithms, often surpassing them in certain scenarios. In Table 5, *Heuristic-v1* performs comparably well on the EU Power Grid Network, maintaining robustness with an average of 0.205911 and a low variance, often surpassing the baseline algorithms in stability.

Specifically, *Heuristic-v1* maintains stable performance with low variance across all network sizes and densities. For example, as shown in Table 3, *Heuristic-v1* outperforms the baseline algorithms in the 100-node network scenario by achieving a lower variance of 0.000069. In Table 4, for edge density $M_0 = 2$, *Heuristic-v1* shows better average robustness, 0.262601, and lower variance than the baseline algorithms.

Heuristic-v2 consistently achieves the best performance across most test cases, demonstrating its superior robustness enhancement capabilities. However, it does not outperform the other algorithms in every scenario. For example, in Table 4, for denser networks with $M_0 = 4$ and $M_0 = 5$, *Heuristic-v2* does not achieve the highest robustness, as it is slightly outperformed by SA and *Heuristic-v3*. Nonetheless, *Heuristic-v2* remains one of the top-performing algorithms overall, excelling in the majority of scenarios.

Heuristic-v3, although not as high-performing as *Heuristic-v2*, consistently outperforms the baseline algorithms. For example, in Table 3, *Heuristic-v3* shows strong performance in larger networks, achieving an average robustness of 0.307550 for 200 nodes and 0.307023 for 300 nodes, outperforming all baseline algorithms. As shown in Table 4, for denser networks with $M_0 = 4$ and $M_0 = 5$, *Heuristic-v3* achieves the highest robustness, with average values of 0.412866 and 0.427933, respectively, demonstrating its effectiveness in denser networks. Similarly, as shown in Table 5, *Heuristic-v3* achieves a robustness of 0.219386 in the EU Power Grid Network, outperforming the baseline algorithms.

To confirm the statistical significance of AutoRNet's performance improvements, we conducted a *Wilcoxon rank-sum test* on the results of all test scenarios. The *p* values in the comparisons between *Heuristic-v1*,

Heuristic-v2, *Heuristic-v3*, and baseline algorithms were all **less than 0.01**, confirming that the differences between the AutoRNet-generated heuristics and baseline methods are statistically significant.

Impact of the AFF and temperature

In traditional EAs, the fitness function directly evaluates solutions within a constrained search space. However, in AutoRNet, the search space consists of heuristics rather than direct solutions, meaning that the fitness evaluation depends on the effectiveness of the generated heuristics rather than the solutions themselves. This fundamental difference affects how fitness evolves over time.

Figure 4 illustrates this effect. In the absence of the AFF, the fitness function strictly enforces degree distribution constraints at all stages. Consequently, the generated heuristics remain confined to the initial seed heuristic, which primarily performs random edge exchanges. This lack of exploration results in a negligible fitness improvement over the generations, leading to a nearly flat fitness curve.

When the AFF is introduced, the fitness function initially relaxes the constraints, allowing for a broader exploration of the heuristic space. This leads to a rapid increase in fitness as the generated heuristics become more effective in enhancing network robustness. However, as evolution progresses, the AFF progressively strengthens the constraints, guiding the heuristics towards solutions that fully satisfy the degree distribution requirements. This constraint tightening results in a gradual decline in fitness, as heuristics that were previously effective under relaxed constraints become less viable under stricter conditions. Eventually, the process stabilizes, with fitness converging to a final plateau at which the heuristics maintain both robustness and structural feasibility. As shown in Fig. 4, the performance of AutoRNet gradually converges after 40 generations. Since increasing the number of generations will raise the cost of calling LLMs, we have set the number of generations to 50.

With respect to the temperature parameter, we evaluated two settings: 1.0 and 0.5. While both configurations ultimately produced heuristics of comparable effectiveness, a temperature of 1.0 led to faster fitness improvement in the early generations. This suggests that increasing the temperature, by increasing the diversity of LLM-generated heuristics, enhances exploration and accelerates convergence towards high-quality heuristics. As the constraints tighten in later generations, both temperature settings converge to similar results, indicating that the initial randomness aids exploration, whereas the final strict constraints ensure adherence to the structural constraints.

These observations highlight a fundamental characteristic of AutoRNet: *fitness evolution is influenced by heuristic exploration rather than direct solution improvement*. Unlike traditional EAs, the fitness of which typically increases monotonically, the observed nonmonotonic fitness progression in Fig. 4 is a result of the AFF's dynamic constraint enforcement. This underscores the importance of balancing heuristic diversity in the early generations with structural feasibility in the later generations to achieve optimal results.

Limitations and future research directions

While AutoRNet demonstrates significant advancements in generating heuristics for network robustness optimization, it faces several limitations that highlight opportunities for future research.

A primary limitation is the reliance on manually curated NOSs and domain-specific prompts, which require expert knowledge to define network properties. While AutoRNet automates heuristic code generation, the quality of these heuristics depends on the expertise embedded in NOSs and prompts, limiting the extent of automation. Another challenge is the computational overhead of using high-capacity LLMs for heuristic generation and repeated fitness evaluations. This is particularly pronounced for large-scale networks, where computational costs can be significant. Additionally, the potential variability in LLM outputs poses a risk of inconsistent or suboptimal heuristics. Although AutoRNet mitigates this through generating multiple heuristic candidates to ensure diversity, variability may still impact heuristic quality in certain cases. Finally, AutoRNet is designed specifically for scale-free network robustness optimization, and its applicability to other combinatorial optimization problems, such as scheduling or clustering, remains untested, limiting its demonstrated scope.

Future research can address these limitations through several promising directions. To reduce dependency on manual prompt design, automated prompt evolution techniques, such as prompt optimization^{30,31} or Retrieval-Augmented Generation³², could dynamically generate or refine prompts or retrieved domain knowledge, minimizing expert intervention. To enhance computational efficiency, model compression techniques, including pruning³³ and knowledge distillation³⁴, can reduce the computational footprint of LLMs while preserving performance. Surrogate models³⁵ could approximate the robustness measure R , reducing the need for expensive fitness evaluations, and distributed computing³⁶ can parallelize the evolutionary process to improve scalability for larger networks. To address LLM output instability and lack of transparency, enhancing explainability and reasoning capabilities through techniques like chain-of-thought prompting³⁷, Self-Taught Reasoner (STaR)^{38–40}, and explainable AI (XAI) methods^{41,42} could improve the consistency, reliability, and interpretability of generated heuristics. XAI techniques can provide insights into why specific optimization strategies are produced, enabling error diagnosis and refinement, while STaR's iterative reasoning can enhance heuristic quality by allowing LLMs to refine outputs over multiple steps. Exploring the application of AutoRNet to other network types, such as random or small-world networks, or broader optimization domains, could further validate its generalizability. These advancements would enhance AutoRNet's practicality, scalability, and robustness for real-world applications.

Conclusion

This paper presents AutoRNet, a novel framework integrating LLMs with EAs to automate the generation of complete heuristics for enhancing scale-free network robustness. Building on the growing field of LLM-driven optimization, AutoRNet reduces the reliance on time-consuming manual heuristic design by leveraging expert-

crafted NOSs and domain-specific prompts, improving efficiency for complex network problems. The approach employs network optimization strategy (NOS)-based variation operations to craft domain-specific LLM prompts and an adaptive fitness function (AFF) to relax hard constraints, improving search effectiveness. Experimental results demonstrate that AutoRNet designs heuristics matching manual complexity, outperforming baseline algorithms on synthetic networks of varying sizes and densities, and a real-world network, at comparable cost. These results advance knowledge by expanding automated heuristic design for real-world applications. Despite its advancements, AutoRNet has several limitations. These include reliance on expert-defined prompts and NOSs, potential variability in LLM outputs, limited generalizability to other domains, and significant computational costs for large networks. The framework relies on high-quality LLM responses, which may introduce variability if the LLM generates suboptimal or inconsistent heuristics. Additionally, the computational cost of fitness evaluations, particularly for large networks, can be significant due to repeated calls to *R*. Scaling AutoRNet to very large real-world networks may require substantial resources, and the lack of transparency in LLM-generated heuristics poses challenges for understanding their decision-making processes. Addressing these challenges through automated prompt optimization, model compression, and broader evaluations, including its potential application to diverse network topologies or optimization problems, will be critical for enhancing AutoRNet's practicality and robustness.

Data availability

The datasets generated during and/or analysed during the current study are available in the GitHub repository, <https://github.com/leonyuhe/AutoRNet>.

Received: 11 November 2024; Accepted: 16 October 2025

Published online: 20 November 2025

References

- Watts, D. J. & Strogatz, S. H. Collective dynamics of 'small-world' networks. *Nature* **393**(6684), 440–442 (1998).
- Barabási, A.-L. & Albert, R. Emergence of scaling in random networks. *Science* **286**(5439), 509–512 (1999).
- Schneider, C. M., Moreira, A. A., Andrade, J. S., Havlin, S. & Herrmann, H. J. Mitigation of malicious attacks on networks. *Proc. Natl. Acad. Sci.* **108**(10), 3838–3841 (2011).
- Albert, R., Jeong, H. & Barabási, A.-L. Error and attack tolerance of complex networks. *Nature* **406**(6794), 378–382 (2000).
- Stojan, T., Javier, M. H., Wynand, W. & Piet, M. Robustness envelopes of networks. *J. Complex Netw.* **1**, 44–62 (2013).
- Buesser, P., Daolin, F., Tomassini, M. Optimizing the robustness of scale-free networks with simulated annealing. In *Proceedings of the 10th International Conference on Adaptive and Natural Computing Algorithms*, Vol. Part II, 2011, pp. 167–176.
- Zhou, M. & Liu, J. A memetic algorithm for enhancing the robustness of scale-free networks against malicious attacks. *Phys. A* **410**, 131–143 (2014).
- Zeng, A. & Liu, W. Enhancing network robustness against malicious attacks. *Phys. Rev. E* **85**, 066130 (2012).
- Mnih, V., et al. Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015).
- Naveed, H., et al., A comprehensive overview of large language models, arXiv preprint [arXiv:2307.06435](https://arxiv.org/abs/2307.06435) (2023).
- Silva, F., Gonçalves, A., Nguyen, S., Vashchenko, D., Glatt, R., Desautels, T., Landajuela, M., Petersen, B., & Faissol, D. Leveraging language models to efficiently learn symbolic optimization solutions. In *Learning and Adaptive Agents (ALA) Workshop at AAMAS*, (2022).
- AhmadiTeshnizi, A., Gao, W., & Udell, M. OptiMUS: Optimization modeling using MIP solvers and large language models, arXiv preprint [arXiv:2310.06116](https://arxiv.org/abs/2310.06116) (2023).
- Romera-Paredes, B. et al. Mathematical discoveries from program search with large language models. *Nature* **625**, 468–475 (2024).
- Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., & Zhang, Q. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *International Conference on Machine Learning (ICML)*, (2024).
- Chan, H. & Akoglu, L. Optimizing network robustness by edge rewiring: a general framework. *Data Min. Knowl. Disc.* **30**, 1395–1425 (2016).
- Holland, J. H. *Adaptation in Natural and Artificial Systems* (MIT Press, Cambridge, MA, 1975).
- Kirkpatrick, S., Gelatt, C. D. & Vecchi, M. P. Optimization by simulated annealing. *Science* **220**, 671–680 (1983).
- Pizzuti, C., & Socievoli, A. A genetic algorithm for improving robustness of complex networks. In *Proceedings of the 2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI)*, 2018, pp. 514–521.
- Tang, X., et al. Transferring robustness for graph neural network against poisoning attacks. In *Proceedings of the 13th International Conference on Web Search and Data Mining*, 2020, pp. 600–608.
- Wang, B., Jia, J., Cao, X., Gong, N. Z. Certified robustness of graph neural networks against adversarial structural perturbation. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1645–1653 (2021).
- Schulhoff, S., et al. The prompt report: A systematic survey of prompting techniques, arXiv preprint [arXiv:2406.06608](https://arxiv.org/abs/2406.06608) (2024).
- Kojima, T., et al. Large language models are zero-shot reasoners, arXiv preprint [arXiv:2205.11916](https://arxiv.org/abs/2205.11916) (2022).
- Brown, T., et al., Language models are few-shot learners, arXiv preprint [arXiv:2005.14165](https://arxiv.org/abs/2005.14165) (2020).
- Wei, J., et al., Chain-of-Thought prompting elicits reasoning in large language models, arXiv preprint [arXiv:2201.11903](https://arxiv.org/abs/2201.11903) (2022).
- Gao, Y., et al. Retrieval-augmented generation for large language models: A survey, arXiv preprint [arXiv:2312.10997](https://arxiv.org/abs/2312.10997) (2023).
- Liu, S., et al. Large language models as evolutionary optimizers, arXiv preprint [arXiv:2310.19046v3](https://arxiv.org/abs/2310.19046v3) (2024).
- Paterson, J., & Ombuki-Berman, B. Optimizing scale-free network robustness with the great deluge algorithm. In *Recent Trends and Future Technology in Applied Intelligence* 434–446 (Springer, 2018).
- Louzada, V., Daolio, F., Herrmann, H. & Tomassini, M. Smart rewiring for network robustness. *J. Complex Netw.* **1**, 150–159 (2013).
- Zhou, Q. & Bialek, J. Approximate model of European interconnected system as a benchmark system to study effects of cross-border trades. *IEEE Trans. Power Syst.* **20**(2), 782–788 (2005).
- Guo, Q., et al. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers, arXiv preprint [arXiv:2309.08532](https://arxiv.org/abs/2309.08532) (2024).
- Ye, H., et al. ReEvo: Large language models as hyper-heuristics with reflective evolution, in: *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- Lewis, P., et al. Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- Han, S., Pool, J., Tran, J., & Dally, W. Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding, arXiv preprint [arXiv:1510.00149](https://arxiv.org/abs/1510.00149) (2015).

34. Hinton, G., Vinyals, O., & Dean, J. Distilling the knowledge in a neural network, in: Proceedings of the NIPS Deep Learning and Representation Learning Workshop (2015).
35. Hao, H., Zhang, X., & Zhou, A. Large language models as surrogate models in evolutionary algorithms: A preliminary study, arXiv preprint [arXiv:2406.10675](https://arxiv.org/abs/2406.10675) (2024).
36. Dean, J. & Ghemawat, S. MapReduce: simplified data processing on large clusters. *Commun. ACM* **51**(1), 107–113 (2008).
37. Wei, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **35**, 24824–24837 (2022).
38. Zelikman, E., Wu, Y., Mu, J., & Goodman, N. D. STaR: Bootstrapping reasoning with reasoning, arXiv preprint [arXiv:2203.14465](https://arxiv.org/abs/2203.14465) (2022).
39. Hosseini, A., Yuan, X., Malkin, N., Courville, A., Sordoni, A., & Agarwal, R. V-STaR: training verifiers for self-taught reasoners, arXiv preprint [arXiv:2402.06451](https://arxiv.org/abs/2402.06451) (2024).
40. Zelikman, E. et al. Quiet-STaR: Language models can teach themselves to think before speaking, arXiv preprint [arXiv:2403.09629](https://arxiv.org/abs/2403.09629) (2024).
41. Ribeiro, M. T., et al. Why should I trust you? Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
42. Samek, W., Wiegand, T., & Müller, K.-R. Explainable artificial intelligence: understanding, visualizing and interpreting deep learning models, arXiv preprint [arXiv:1708.08296](https://arxiv.org/abs/1708.08296) (2017).

Author contributions

H. Y. and J. L. figured out the idea. H. Y. implemented the algorithms, conducted the major experiments, and wrote the main manuscript text. J. L. improved the main manuscript text.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1038/s41598-025-25031-2>.

Correspondence and requests for materials should be addressed to H.Y.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025